

从SBI到stdio

承接上一节的问题，我们需要在“一无所有”的环境中，创建一个能用的 `cprintf` 函数。

须知

如果我们在 Linux 下运行一个 C 程序，需要格式化输出，那么大一的同学都知道我们应该 `#include<stdio.h>`。于是我们在 `kern/init/init.c` 也这么写一句。**且慢！** 在 Linux 下，当我们调用 C 语言标准库的函数时，实际上依赖于 `glibc` 提供的运行时环境，也就是一定程度上依赖于操作系统提供的支持。那么这样的操作在逻辑上就是不通顺的，构成了一个**鸡生蛋蛋生鸡**的过程，你不能在开发一个操作系统的时候还要依赖另一个操作系统提供的代码环境支持（注意，这里的支持不是指虚拟机、模拟器等，后面会学到，标准库的`printf`本质上就是调用了操作系统内核提供的接口）。

怎么办呢？只能自己动手，丰衣足食。解决问题的起点，是RISC-V架构下的机器态固件——OpenSBI。QEMU 内置的 OpenSBI 固件为我们提供了一个最原始的“输出一个字符”的接口。我们的任务就是抓住这个原始的接口，像搭积木一样，层层封装，最终构建出我们需要的、功能强大的 `cprintf` 函数

什么是 OpenSBI?

您可以将其理解为一套预先安装在机器（M模式）上的标准函数库。这套库提供了一些基础服务，比如设置定时器、发送处理器间中断（IPI），以及我们最需要的控制台输入输出。

然而，调用这些函数不能像普通函数那样使用 `call` 指令。因为我们的内核运行在 `S` 模式，而 `SBI` 服务运行在更高的 `M` 模式。跨越这种特权级别的调用，需要使用特殊的指令——`ecall`（Environment Call）。

须知 `ecall`

`ecall` 指令是 RISC-V 中用于实现受控的权限提升的关键指令。

当在 `U` 模式（用户态）执行 `ecall`，会触发异常，从而陷入到 `S` 模式（内核态）。这是系统调用的底层机制。

当在 `S` 模式（内核态）执行 `ecall`，会触发异常，从而陷入到 `M` 模式（机器态）。这正是我们调用 `OpenSBI` 服务的方式。

如何调用 SBI 服务？

通过 `ecall` 调用 SBI 服务，需要遵循一个明确的调用约定。这个过程类似于在一个预定义的表格中查找一个功能号，然后按照固定的规则传递参数（后面会学习到这就是系统调用的调用格式），最后执行一个特殊指令来触发它。

1. 指定服务编号：将想要调用的 SBI 功能编号（例如，`SBI_CONSOLE_PUTCHAR = 1`）放入指定的寄存器（通常是 `a7` 或 `x17`）。
2. 传递参数：根据 RISC-V 的函数调用约定（Calling Convention），将参数放入寄存器 `a0`，`a1`，`a2`（即 `x10`，`x11`，`x12`）。
3. 执行调用：执行 `ecall` 指令。CPU 会 trap 到 M 模式，由 OpenSBI 固件处理请求。
4. 获取返回值：处理完成后，OpenSBI 会将返回值放入 `a0`（`x10`）寄存器，然后返回。

为什么需要内联汇编？

在 C 语言中，我们无法直接执行 `ecall` 这样的特定指令，也无法精确控制哪个变量放入哪个寄存器。因此，我们必须借助内联汇编（Inline Assembly）来“手动”完成上述步骤，将底层指令的调用封装成一个对 C 语言友好的函数。

下面的代码实现了最核心的 SBI 调用封装：

```

// libs/sbi.c
#include <sbi.h>
#include <defs.h>

// SBI 功能编号清单
uint64_t SBI_SET_TIMER = 0;
uint64_t SBI_CONSOLE_PUTCHAR = 1;
uint64_t SBI_CONSOLE_GETCHAR = 2;
// ... 其他功能编号

// sbi_call - 通用的 SBI 调用函数
// @sbi_type: SBI 功能编号
// @arg0, arg1, arg2: 传递给 SBI 服务的参数
// 返回值: SBI 服务返回的结果
uint64_t sbi_call(uint64_t sbi_type, uint64_t arg0, uint64_t arg1, uint64_t arg2) {
    uint64_t ret_val;

    __asm__ volatile (
        // 1. 将功能编号和参数放入指定的寄存器
        "mv x17, %[sbi_type]\n" // 功能编号 -> x17 (a7)
        "mv x10, %[arg0]\n"      // arg0 -> x10 (a0)
        "mv x11, %[arg1]\n"      // arg1 -> x11 (a1)
        "mv x12, %[arg2]\n"      // arg2 -> x12 (a2)

        // 2. 执行 ecall 指令, 发起调用
        "ecall\n"

        // 3. 将返回值 (在 x10/a0 中) 移动到 C 变量 ret_val 中
        "mv %[ret_val], x10"

        // 输出操作数: 将汇编的结果输出到C变量ret_val
        : [ret_val] "=r" (ret_val)
        // 输入操作数: 将C变量sbi_type, arg0, arg1, arg2的值作为输入传给汇编
        : [sbi_type] "r" (sbi_type), [arg0] "r" (arg0), [arg1] "r" (arg1), [arg2] "r"
        // 告知编译器: 内联汇编可能会读取或写入内存, 防止编译器优化时出错
        : "memory"
    );
    return ret_val;
}

// 基于通用的 sbi_call, 封装出专用的字符输出函数
void sbi_console_putchar(unsigned char ch) {

```

```
sbi_call(SBI_CONSOLE_PUTCHAR, ch, 0, 0);  
}
```

这样我们就可以通过 `sbi_console_putchar()` 来输出一个字符。接下来我们要做的事情就像月饼包装，把它封了一层又一层。

`console.c` 只是简单地封装一下

```
// kern/driver/console.c  
#include <sbi.h>  
#include <console.h>  
  
void cons_putc(int c) { sbi_console_putchar((unsigned char)c); }
```

`stdio.c` 里面实现了一些函数，注意我们已经实现了ucore版本的puts函数: `cputs()`

```
// kern/libs/stdio.c
#include <console.h>
#include <defs.h>
#include <stdio.h>

/* HIGH level console I/O */

/* *
 * cputch - writes a single character @c to stdout, and it will
 * increace the value of counter pointed by @cnt.
 * */
static void cputch(int c, int *cnt) {
    cons_putc(c);
    (*cnt)++;
}

/* cputchar - writes a single character to stdout */
void cputchar(int c) { cons_putc(c); }

int cputs(const char *str) {
    int cnt = 0;
    char c;
    while ((c = *str++) != '\0') {
        cputch(c, &cnt);
    }
    cputch('\n', &cnt);
    return cnt;
}
```

我们还在 `libs/printfmt.c` 实现了一些复杂的格式化输入输出函数。最后得到的 `cprintf()` 函数仍在 `kern/libs/stdio.c` 定义，功能和C标准库的 `printf()` 基本相同。

可能你注意到我们用到一个头文件 `defs.h`，我们在里面定义了一些有用的宏和类型

```

// libs/defs.h
#ifndef __LIBS_DEFS_H__
#define __LIBS_DEFS_H__
...
/* Represents true-or-false values */
typedef int bool;
/* Explicitly-sized versions of integer types */
typedef char int8_t;
typedef unsigned char uint8_t;
typedef short int16_t;
typedef unsigned short uint16_t;
typedef int int32_t;
typedef unsigned int uint32_t;
typedef long long int64_t;
typedef unsigned long long uint64_t;
...
/* *
 * Rounding operations (efficient when n is a power of 2)
 * Round down to the nearest multiple of n
 * */
#define ROUNDDOWN(a, n) ({ \
    size_t __a = (size_t)(a); \
    (typeof(a))(__a - __a % (n)); \
})
...
#endif

```

printfmt.c 还依赖一个头文件 `riscv.h` ,这个头文件主要定义了若干和riscv架构相关的宏,尤其是将一些内联汇编的代码封装成宏,使得我们更方便地使用内联汇编来读写寄存器。当然这里我们还没有用到它的强大功能。

```
// libs/riscv.h
...
#define read_csr(reg) ({ unsigned long __tmp; \
    asm volatile ("csrr %0, " #reg : "=r"(__tmp)); \
    __tmp; })
//通过内联汇编包装了 csrr 指令为 read_csr() 宏
#define write_csr(reg, val) ({ \
    if (__builtin_constant_p(val) && (unsigned long)(val) < 32) \
        asm volatile ("csrw " #reg ", %0" :: "i"(val)); \
    else \
        asm volatile ("csrw " #reg ", %0" :: "r"(val)); })
...
```

到现在，我们已经看过了一个最小化的内核的各个部分，虽然一些部分没有逐行细读，但我们也知道它在做什么。但一直到现在我们还没进行过编译。下面就把它编译一下跑起来。